Utilizando RPC com Java

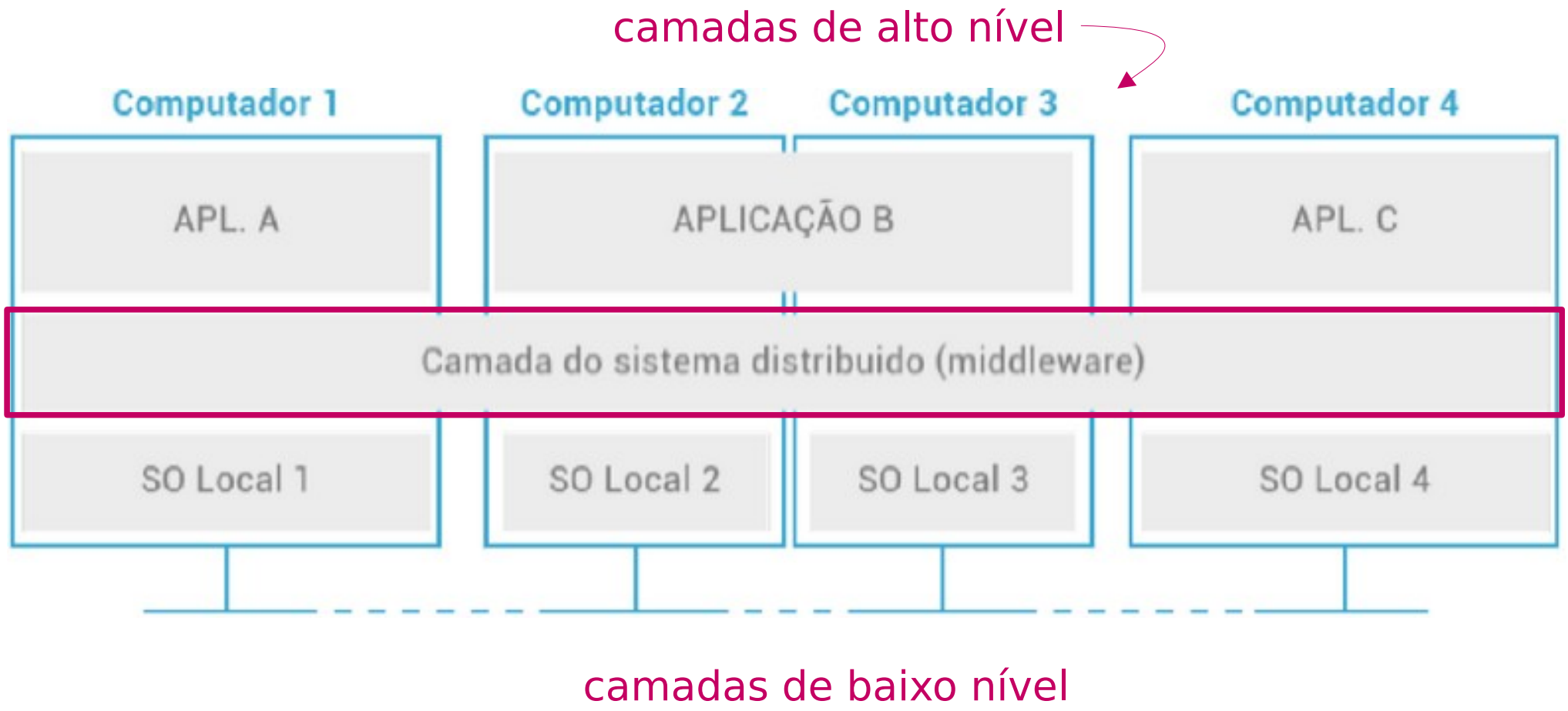
Sistemas Distribuídos

Eduardo Furlan Miranda

2025-11-02

Adaptado de: PEREIRA, C. S. Sistemas Distribuídos.
Londrina: EDE SA, 2019. ISBN 978-85-522-1443-4.

middleware: camada de abstração



RPC

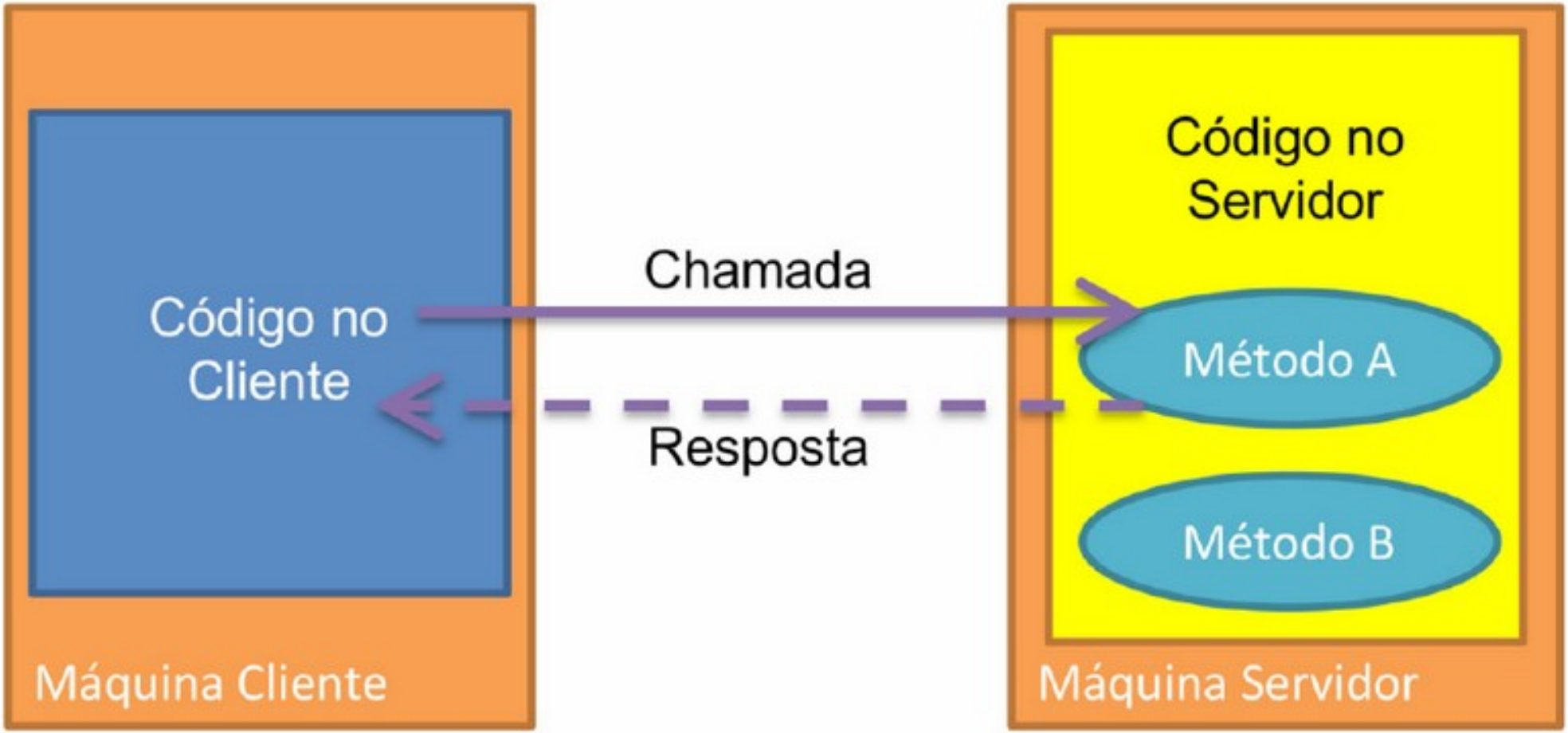
- Forma de comunicação entre máquinas
- Permite chamada de métodos de alto nível, pelas aplicações
- O cliente executa métodos no servidor, como se tivesse sido implementado localmente
- Possui mais recursos que Socket
- Envolve as 7 camadas OSI (alta granulariadade)

Modelos de comunicação entre máquinas

Modelo de Comunicação	Implementado através de
RPC (*Remote Procedure Call*)	Linguagens de programação estruturadas.
RMI (*Remote Method Invocation*)	Linguagens de programação orientadas a objeto.
MOM (*Message Oriented Midleware*)	Linguagens de programação para web.

- Existem vários *frameworks* que podem ser utilizados
  - Java: JRMI, JMS
  - C#: .NET, Akka.net
  - Python: Pyro, RPyC
  - JavaScript: Jason, Dnode

Componentes de uma comunicação via RPC



Exemplo

- Usando Java com JRMI
- Escrevemos um método remoto executado pelo cliente
- O método recebe um nome e converte para maiúsculas
- Iniciamos com uma “interface” em Java para declarar os métodos

Interface.java

herda as propriedades da classe

```
import java.rmi.Remote;

public interface Interface extends Remote {

    public String converterParaMaiuscula(String palavra) throws Exception;

}
```

para falhas de execução em tempo de execução

Servidor.java

comunicação 1:1

```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
import java.rmi.server.UnicastRemoteObject;

public class Servidor extends UnicastRemoteObject implements Interface {

    public static void main(String[] args) {
        try {
            System.setProperty("java.rmi.server.hostname", "10.116.201.48");
            LocateRegistry.createRegistry(1099);
            Naming.rebind("//10.116.201.48/obj", new Servidor());
        } catch (Exception e) {
            System.err.println("Falha ao iniciar o servidor!");
        }
    }

    public Servidor() throws Exception {
    }

    @Override
    public String converterParaMaiuscula(String palavra) throws Exception {
        return palavra.toUpperCase();
    }
}
```

“obj” é um nome dado

- “UnicastRemoteObject” informa qual tipo de comunicação será realizada entre servidor-cliente
  - *unicast* : comunicação 1 para 1
- Porta padrão JRMI: 1099 TCP
- “Naming.rebind”: define os parâmetros para criação do objeto que será compartilhado remotamente (a classe “Servidor()”)
  - “obj” é um nome dado
- “Servidor()”: é um construtor vazio, com “throws Exception”

Cliente.java

O método “lookup” pega a referência ao objeto. Pode-se utilizar como se existisse localmente.

```
import java.rmi.Naming;
import java.util.Scanner;

public class Cliente {

    public static void main(String[] args) throws Exception {

        Interface objetoRemoto = (Interface) Naming.lookup("//10.116.201.48/obj");

        System.out.println(objetoRemoto.converterParaMaiuscula(
                                        new Scanner(System.in).nextLine())
                        );
    }
}
```

Exemplo para o Colab

```
# Instala o Java
!apt-get install -y openjdk-11-jdk
```

```
%%writefile Interface.java
import java.rmi.Remote;
import java.rmi.RemoteException; // É boa prática usar RemoteException

public interface Interface extends Remote {
    public String converterParaMaiuscula(String palavra) throws RemoteException;
}
```

```
%%writefile Servidor.java
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;

public class Servidor extends UnicastRemoteObject
        implements Interface, Runnable {

    // Construtor
    public Servidor() throws RemoteException {
        super(); // Chama o construtor de UnicastRemoteObject
    }



// Implementação do método da interface
@Override
public String converterParaMaiuscula(String palavra)
        throws RemoteException {
    System.out.println("[Servidor] Recebeu: " + palavra);
    String resultado = palavra.toUpperCase();
    System.out.println("[Servidor] Enviando: " + resultado);
    return resultado;
}
```

```java
    // Método que será executado pela Thread do Servidor
    @Override
    public void run() {
        try {
            // 1. Inicia o RMI Registry na porta 1099
            //    (dentro deste mesmo programa)
            LocateRegistry.createRegistry(1099);
            System.out.println(
                "[Servidor] RMI Registry iniciado na porta 1099.");

            // 2. Registra o objeto (esta própria instância)
            //    Usamos 'localhost' porque está na mesma máquina
            Naming.rebind("//localhost/obj", this);

            System.out.println(
                "[Servidor] Servidor pronto e registrado como " +
                "'//localhost/obj'.");
            System.out.println(
                "[Servidor] Aguardando chamadas do cliente...");

        } catch (Exception e) {
            System.err.println("[Servidor] Falha ao iniciar: " +
                e.getMessage());
            e.printStackTrace();
        }
    }
}
```

```
%%writefile Cliente.java
import java.rmi.Naming;
import java.util.Scanner;
import java.rmi.RemoteException;

public class Cliente implements Runnable {

    // Método que será executado pela Thread do Cliente
    @Override
    public void run() {
        try {
            System.out.println(
                "[Cliente] Procurando pelo objeto remoto " +
                "'//localhost/obj'...");

            // 1. Procura o objeto no RMI Registry (em localhost)
            Interface objetoRemoto = (Interface) Naming.lookup(
                "//localhost/obj");

            System.out.println("[Cliente] Objeto remoto encontrado!");
```

```
            // 2. Pega a entrada do usuário
            Scanner scanner = new Scanner(System.in);
            System.out.print(
                "[Cliente] Digite uma palavra para converter: ");
            String palavra = scanner.nextLine();

            // 3. Chama o método remoto
            String resultado =
                objetoRemoto.converterParaMaiuscula(palavra);

            // 4. Imprime o resultado
            System.out.println("\n--- RESULTADO DA CONVERSÃO ---");
            System.out.println(resultado);
            System.out.println("-------------------------------\n");

            scanner.close();

        } catch (Exception e) {
            System.err.println("[Cliente] Falha ao executar: " +
                e.getMessage());
            e.printStackTrace();
        }
    }
}
```

```
%%writefile TestadorRMI.java
import java.rmi.RemoteException;

public class TestadorRMI {
    public static void main(String[] args) {
        try {
            // 1. Cria a implementação do Servidor
            Servidor servidor = new Servidor();

            // 2. Inicia o Servidor em uma nova Thread
            Thread threadServidor = new Thread(servidor);
            threadServidor.start(); // Chama o método Servidor.run()
            System.out.println("[Main] Thread do Servidor iniciada.");

            // 3. Pausa rápida (1 seg) para dar tempo ao servidor
            //    de iniciar e se registrar
            //    (Em apps reais, usaria sincronização)
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {}
```

```
        // 4. Cria e inicia o Cliente em outra Thread
        Cliente cliente = new Cliente();
        Thread threadCliente = new Thread(cliente);
        threadCliente.start(); // Chama o método Cliente.run()
        System.out.println("[Main] Thread do Cliente iniciada.");

    } catch (RemoteException e) {
        System.err.println(
            "[Main] Erro ao criar o servidor: " +
            e.getMessage());
    }
  }
}
```

```
!javac *.java
```

```
!java TestadorRMI
```

```
[Main] Thread do Servidor iniciada.
[Servidor] RMI Registry iniciado na porta 1099.
[Servidor] Servidor pronto e registrado como '//localhost/obj'.
[Servidor] Aguardando chamadas do cliente...
[Cliente] Procurando pelo objeto remoto '//localhost/obj'...
[Main] Thread do Cliente iniciada.
[Cliente] Objeto remoto encontrado!
[Cliente] Digite uma palavra para converter: teste
[Servidor] Recebeu: teste
[Servidor] Enviando: TESTE

--- RESULTADO DA CONVERSÃO ---
TESTE
-----------------------------
```